

A Physics-Informed Neural Network Forward Solver for the Rough-Volatility Calibration Inverse Problem: Dimension Scaling on the Markovian-Lifted PDE and a Fast Warm-Start Hybrid

Aviral Sharma^{a,*}, Kamal Preet Singh^b, Vipul Sharma^c

^a*Chandigarh University, Chandigarh, India*

^b*SGT University, Gurugram, India*

^c*Independent Researcher, India*

Abstract

Calibrating a rough-volatility model to an option surface is an inverse problem whose fractional, non-Markovian variance admits no finite-dimensional forward partial differential equation (PDE). A Markovian lift approximates the kernel by n exponentials and recovers an $(n+1)$ -dimensional forward PDE, which we solve with a physics-informed neural network (PINN) trained on its residual. Measured against an independent lifted Monte Carlo at the same n , and separated from the lift’s own approximation error, the network’s solve-error shows no statistically detectable growth as n rises from 4 to 32 (log-log slope 0.09 ± 0.08 , eight seeds), whereas a naive full-tensor finite-difference grid—an illustrative bound, not a tuned solver—is infeasible by $n=8$. The solve/lift decomposition shows whether to add factors or improve the network. A parametric extension is too inaccurate to calibrate alone, but as a warm start for a short exact-model refinement it recovers full calibration quality at 8.8 times fewer expensive evaluations. On live Bitcoin options from Deribit, the surface pins the correlation ($\rho \approx -0.43$) but identifies the Hurst exponent only to an interval, $H \in [0.04, 0.30]$, well inside the rough regime; a realised-volatility estimate using no option data falls inside it. The fit is 113 vol bp vega-weighted and 416 unweighted, the latter dominated by options under two weeks; beyond two months it is 52 vol bp.

*Corresponding author

Email address: aviral198765@gmail.com (Aviral Sharma)

Keywords: inverse problems, physics-informed neural networks, rough volatility, Markovian lift, model calibration, curse of dimensionality, scientific machine learning

1. Introduction

Model calibration is the canonical inverse problem of mathematical finance: given a finite set of noisy, arbitrage-constrained option prices, recover the parameters of a forward model that reproduces them. Rough-volatility models make this inverse problem unusually severe, because their *forward* problem is already pathological. Motivated by the empirical roughness of realised variance [1, 2] and the steep short-maturity skew that classical stochastic-volatility models such as Heston’s [3] cannot reproduce, rough-volatility models drive the variance with a fractional noise whose Hurst exponent $H < 1/2$. That single fact—whose lineage runs back to long-memory volatility [4]—makes the variance process neither Markovian nor a semimartingale, so there is no finite-dimensional forward PDE to solve, let alone invert [1, 5].

The standard engineering escape is the *Markovian lift*: approximate the fractional kernel by a sum of n exponentials and recover a genuine $(n+1)$ -dimensional forward PDE [6, 7]. This turns an ill-posed forward problem into a well-posed but high-dimensional one, and pushes the difficulty of the inverse problem onto the cost of repeatedly solving that high-dimensional forward PDE inside a calibration loop—once per candidate parameter vector, at every iteration of the outer optimiser. Any AI method proposed as a forward operator for this inverse problem must therefore be judged on two separate questions: how accurately does it solve the forward PDE it is handed, and how does that accuracy behave as the lifted dimension n —the knob that trades model fidelity against forward-problem cost—is turned up. Both questions are usually left unanswered when a neural network is dropped into a calibration pipeline as a black-box pricer.

We answer them directly for a physics-informed neural network (PINN) forward operator. Rather than laying down a grid, a PINN represents the solution as a neural network and trains it to satisfy the PDE at randomly sampled collocation points [8], sidestepping the exponential blow-up that kills finite differences in high dimensions, in the spirit of the deep Galerkin and deep BSDE solvers [9, 10]. Because it is differentiable in its inputs, the same

trained network doubles as a differentiable forward operator for gradient-based calibration once the parameters are added as additional inputs. Our concrete contributions are as follows.

1. **A convergence study of the AI forward operator.** The PINN’s own solve-error shows no statistically detectable growth as the lifted dimension climbs from 4 to 32, and its cost per iteration is flat, while a naive full-tensor finite-difference grid (50^{n+1} points) is infeasible by $n=8$. We identify the four training choices without which this does not hold (Section 4).
2. **An error decomposition that isolates what limits the inverse problem.** We split the pricing error into the network’s solve-error, against a lifted Monte Carlo at the same n , and the lift-error, against the exact Fourier forward map. Their crossover gives a calibration- and maturity-dependent rule for whether to add factors or improve the network.
3. **A differentiable forward operator inside the calibration.** A parametric extension conditioned on the five rough-Heston parameters is too inaccurate to calibrate alone, but as a warm start for a short exact-model refinement it recovers from-scratch quality at 8.8 times fewer expensive evaluations, whichever parameter box it was trained on.
4. **A real-market solution of the inverse problem.** On live Deribit Bitcoin options the surface identifies the correlation sharply and the Hurst exponent only to an interval well inside the rough regime. We report that non-identification, its mechanism and its consequences for risk functionals, rather than a point estimate the data do not support.

We are explicit about what is *not* new. Papapantoleon and Rou [11] solved the lifted rough-Heston forward PDE with a mesh-free network before us. Our contribution is a residual PINN forward operator characterised specifically as a component of the calibration inverse problem: a quantitative convergence-versus-dimension study, an error decomposition that separates network error from model error, a real-market inverse-problem solution, and the warm-start hybrid.

2. Related work

Closest prior work. Papapantoleon and Rou [11] price European options in rough-diffusion models by recasting the forward pricing PDE as an energy-

minimisation problem and approximating it in a time-stepping deep gradient flow, demonstrated on the lifted Heston model. Their method is the closest to ours in target and spirit; it differs in mechanism (time-stepping variational scheme versus residual PINN) and does not study forward-operator dimension scaling, error decomposition, or the calibration inverse problem downstream.

Neural forward operators for rough volatility. Jacquier and Žurič [12] cast the path-dependent pricing equations as backward SDEs and solve them with a random-feature network, reducing training to a least-squares regression. They share the forward target but not the mechanism, and neither dimension scaling nor the inverse problem is studied.

AI methods for the calibration inverse problem. The dominant approach for fast rough-volatility calibration is the supervised surrogate of Bayer et al. [13] and Horvath et al. [14]: a network trained offline on precomputed prices or implied volatilities, used as a fast forward map inside the calibration loop. It is purely data-driven; the network never sees the governing equation, carries no PDE consistency, and—because it is not derived from a physics-informed forward solve—cannot be decomposed into a network-error and a model-error term the way ours can, which is precisely the diagnostic an inverse-problem user needs to trust or distrust a calibrated fit.

PINN training pathologies and remedies. PINNs are well-known to fail as forward operators when loss terms sit on different scales or temporal causality is violated [15, 16]. The two stabilisers we use to keep the forward operator convergent and stable—self-adaptive homoscedastic-uncertainty weights [17, 18] and a causal τ -curriculum [19]—directly address these failure modes.

Foundational methods. We build on PINNs [8] and deep PDE solvers [9, 10]; on rough volatility [1, 5] and its multifactor Markovian lift [6, 7]; and on classical transform pricing [3, 20] for our independent, non-lifted forward-model ground truth.

3. Problem formulation

3.1. Rough Heston and the Markovian lift

Under rough Heston the spot price S and its instantaneous variance V satisfy

$$dS_t = \sqrt{V_t} S_t dW_t^S, \quad (1)$$

$$V_t = V_0 + \int_0^t K(t-s) \lambda(\theta - V_s) ds + \int_0^t K(t-s) \nu \sqrt{V_s} dW_s^V, \quad (2)$$

where W^S and W^V are Brownian motions with correlation ρ , while $K(t) = t^{\alpha-1}/\Gamma(\alpha)$ is the fractional kernel with $\alpha = H + \frac{1}{2}$ and $H < \frac{1}{2}$ [1, 5]. The variance is a Volterra process: non-Markovian, non-semimartingale, and admitting no finite-dimensional forward PDE. The parameter vector $(H, V_0, \theta, \lambda, \nu, \rho)$ is exactly what the calibration inverse problem must recover from market data; without a forward PDE there is no forward operator to embed inside that inverse problem at all. We fix this notation once, because the paper uses two different subsets of it: the full six-parameter vector is what the market calibration of Section 7.4 recovers, whereas the parametric forward operator of Section 7.6 conditions on the five parameters $(H, \nu, \rho, \lambda, \theta)$ that enter the PDE coefficients, holding V_0 (a state, not a coefficient) and the maturity fixed at training time.

The Markovian lift replaces K with $K(t) \approx \sum_{i=1}^n c_i e^{-x_i t}$, introducing n Ornstein–Uhlenbeck factors U^i ,

$$\begin{aligned} dU_t^i &= [-x_i U_t^i + \lambda(\theta - V_t)] dt + \nu \sqrt{V_t^+} dW_t^V, & U_0^i &= 0, \\ V_t &= V_0 + \sum_{i=1}^n c_i U_t^i, \end{aligned} \quad (3)$$

where $V_t^+ = \max(V_t, 0)$ [6, 7]. Two features of (3) drive everything that follows. First, the factors differ *only* in their mean-reversion speed x_i : they share the same drift source $\lambda(\theta - V_t)$, the same diffusion coefficient $\nu \sqrt{V_t^+}$ and, critically, the same Brownian increment dW_t^V . That common driver is why the generator in (4) below carries a full (i, j) double sum rather than a diagonal one; the derivation is given in Appendix A. Second, the speeds (x_i) span roughly ten orders of magnitude at small H , which is the source of the stiffness that dictates our numerical schemes (Section 5). The weights

(c_i, x_i) are placed on a geometric quadrature grid [6]; the relative L^2 kernel error falls monotonically from 25.6% at $n=5$ to 0.39% at $n=100$, so raising n genuinely buys forward-model fidelity, at the cost of raising the forward-problem’s dimension by the same n .

3.2. The lifted forward PDE

The discounted European call price $P(t, x, u)$, where $x = \log S$ and $u = (u_1, \dots, u_n)$ collects the factor values, satisfies the $(n+1)$ -dimensional backward Kolmogorov PDE

$$\begin{aligned} 0 = & P_t + \left(r - \frac{1}{2}V\right) P_x + \frac{1}{2}V P_{xx} + \sum_{i=1}^n [-x_i u_i + \lambda(\theta - V)] P_{u_i} \\ & + \frac{1}{2}\nu^2 V \sum_{i,j=1}^n P_{u_i u_j} + \rho\nu V \sum_{i=1}^n P_{xu_i} - rP, \end{aligned} \quad (4)$$

with terminal condition $P(T, x, u) = \max(e^x - K, 0)$. Because all factors share the single Brownian motion W^V , the full double sum over (i, j) and the $(n+1)$ -dimensional cross term both appear; the PDE dimension is exactly $n+1$. At $n=4$ this is a five-dimensional forward problem; at $n=32$ it is thirty-three. Solving (4) for a single candidate parameter vector is one forward-model evaluation inside the outer calibration loop; a naive optimiser needs hundreds to thousands of these per fit (Section 6).

4. The PINN forward operator: architecture and training

4.1. Baseline network

We represent the forward map as a fully connected multilayer perceptron (MLP). Every hidden unit uses the tanh activation, which provides the smooth, high-order derivatives needed for automatic differentiation of the PDE residual; tanh is standard for PINN residual training and avoids the dead-neuron pathology of ReLU in this high-derivative setting [8]. All weight matrices are initialised with Xavier normal initialisation [21] and all biases with zeros.

The network maps $(\tau, x, u_1, \dots, u_n) \in \mathbb{R}^{2+n}$ to a scalar correction. For the single-point forward operator (parameters fixed) we use width $W=64$ and depth $D=4$ hidden layers; for the parametric forward operator used inside the inverse problem (Section 7.6) we increase to $W=96$, $D=5$. Table 1 summarises all architectural and training hyperparameters.

Table 1: Architecture and training hyperparameters of the PINN forward operator. Values in parentheses are for the parametric extension (Section 7.6).

Component	Value
<i>Architecture</i>	
Network type	Fully connected MLP
Hidden layers	4 (5 for parametric)
Neurons per layer	64 (96 for parametric)
Activation	tanh (all hidden layers)
Weight initialisation	Xavier normal
Bias initialisation	Zero
Input dimension	$2 + n$ for single-point; $2 + n + 5$ for parametric
Output dimension	1 (price correction N_θ)
Floating-point precision	<code>float64</code> (PyTorch)
<i>Optimisation</i>	
Optimiser	Adam [22]
Initial learning rate	10^{-3}
LR schedule	Cosine annealing to 0 over all iterations
Gradient clipping	ℓ_2 norm ≤ 1.0
Training iterations	10,000–20,000 (single-point); 20,000–30,000 (parametric)
<i>Collocation sampling (per iteration)</i>	
Interior collocation	2,000 uniform + 500 near-strike (total 2,500)
Boundary points	400 (split OTM / ITM)
Log-moneyness domain	$[\log K - 1.2, \log K + 1.2]$
Factor sampling	MC-empirical mean / std (Section 4.4)
Tail coverage fraction	20% of u -points drawn at $2 \times$ std
<i>Loss weighting</i>	
Adaptive scheme	Learnable log-variance (homoscedastic uncertainty)
Terms balanced	PDE residual MSE, OTM boundary MSE, ITM boundary MSE
<i>τ-curriculum</i>	
Initial horizon	$\tau_0 = 0.15 T$
Ramp fraction	Linear growth to T over first 50% of iterations
<i>Compute environment (every run in this paper)</i>	
Hardware	Intel Core i5-13450HX CPU, 16 logical cores, 15.7 GB RAM; no GPU
Software	PyTorch 2.12.1 (10 threads), NumPy 2.5.0, SciPy 1.18.0; full list in Supplementary Table S1

4.2. Hard-constrained initial-condition ansatz

We train in time-to-maturity $\tau = T - t$ so the terminal condition becomes an initial condition at $\tau=0$. Rather than penalise it softly, we enforce it exactly through the ansatz

$$P(\tau, x, u) = P_{\text{base}}(\tau, x) + \tau N_{\theta}(\tau, x, u), \quad (5)$$

where N_{θ} is the network output. The τ prefactor forces the learned correction to vanish at $\tau=0$ automatically, so the network never fights the initial condition. Without this, a soft penalty for the terminal payoff oscillates against the PDE residual and the forward operator fails to converge.

4.3. Integrated-variance baseline anchor

The baseline P_{base} is a Black–Scholes call price. A critical design choice is what volatility it is anchored at. Anchoring at the *spot* variance V_0 leaves a systematic level bias: the factors mean-revert from V_0 toward θ over the life of the option, so the model’s actual expected variance is higher than V_0 . We anchor instead at the model’s expected *integrated* variance

$$w(\tau) = \int_0^{\tau} \mathbb{E}[V_s] ds, \quad (6)$$

computed by a short pre-flight Monte Carlo (the same simulation used for factor collocation). The baseline then uses $\sigma(\tau) = \sqrt{w(\tau)/\tau}$, which correctly captures the mean-reversion effect; the network learns only the residual stochastic-volatility correction. This single change removed a uniform ~ 120 volbp level bias we observed with the V_0 anchor—an error that would otherwise propagate directly into every inverse-problem fit using this forward operator.

4.4. MC-informed factor collocation

The factor values u at a given maturity τ are not centred at zero: they mean-revert to a distribution set by the model dynamics. Sampling from a fixed symmetric box centred at zero causes the network to expend capacity on regions the dynamics never visit, and—critically—to under-cover the region governing the put-wing skew. Before training we run a short lifted Monte Carlo simulation (4,000 paths, 300 time steps) to estimate the mean and standard deviation of each factor at the target maturity; training collocation is then drawn from $\mathcal{N}(\hat{\mu}_i, \hat{\sigma}_i)$. An additional 20% of collocation points are

drawn at twice the estimated standard deviation to extend tail coverage. This change cut smile RMSE by roughly 45% (from ~ 210 to ~ 114 vol bp) relative to symmetric-box sampling.

4.5. Self-adaptive loss weighting

Three loss terms must be balanced at each training step: the PDE-residual MSE $\mathcal{L}_{\text{PDE}}$, and the MSEs at the deep-OTM ($P \rightarrow 0$) and deep-ITM ($P \rightarrow S - Ke^{-r\tau}$) boundaries. Fixed-weight balancing is brittle: the ITM boundary target is $O(1)$ while the OTM target is $O(0)$, so a single lumped weight over-emphasises the ITM term. We adopt the learnable homoscedastic-uncertainty scheme of Kendall et al. [17] in the self-adaptive spirit of McClenny and Braga-Neto [18]: each term $\mathcal{L}_k$ is assigned a learnable log-variance s_k and the objective is

$$\mathcal{L} = \sum_{k \in \{\text{PDE}, \text{OTM}, \text{ITM}\}} [e^{-s_k} \mathcal{L}_k + s_k]. \quad (7)$$

The s_k are optimised jointly with the network weights; large or noisy terms are down-weighted automatically. We never hand-tune a boundary weight.

4.6. Causal τ -curriculum

Forward operators that train simultaneously across all time-to-maturities can be pulled toward spurious solutions at long τ before the short- τ region has converged, because the residual at long τ depends causally on the short- τ solution [19]. We address this with a causal curriculum: training begins at $\tau_{\max} = 0.15T$ and linearly ramps to the full T over the first half of training. The short- τ solution is therefore established before the long- τ residual can perturb it.

4.7. Training algorithm

Algorithm 1 summarises the full training procedure for the forward operator.

5. Evaluation methodology: trusting the forward operator

5.1. Two independent ground truths

An AI forward operator used inside an inverse problem is only as trustworthy as its own error against the true forward map. We hold the network to two independent references.

Algorithm 1 Training the PINN forward operator for the lifted rough-Heston PDE

Require: Model parameters $(V_0, \lambda, \theta, \nu, \rho)$; lift $(c_i, x_i)_{i=1}^n$; maturity T ; hyperparameters in Table 1

- 1: Initialise MLP N_θ (Xavier normal); learnable $s = [s_1, s_2, s_3] \leftarrow 0$
- 2: **Pre-flight MC:** simulate lifted SDE, record $(\hat{\mu}_i, \hat{\sigma}_i)$ for each factor and term structure $w(\tau)$ ▷ Sections 4.4, 4.3
- 3: **for** iteration $t = 1, \dots, N_{\text{iter}}$ **do**
- 4: $\tau_{\text{max}} \leftarrow \min(T, 0.15T + \frac{0.85T \cdot t}{0.5N_{\text{iter}}})$ ▷ τ -curriculum, Sec. 4.6
- 5: Sample interior: $\{(\tau_j, x_j, u_j)\}$ from $\text{Unif}([0, \tau_{\text{max}}]) \times \text{Unif}(x_{\text{lo}}, x_{\text{hi}}) \times \mathcal{N}(\hat{\mu}, \hat{\sigma})$
- 6: Sample boundary: $\{(\tau_j^{\text{bc}}, x_{\text{lo/hi}}, u_j^{\text{bc}})\}$
- 7: $\hat{P} \leftarrow P_{\text{base}}(\tau, x; w) + \tau N_\theta(\tau, x, u)$ ▷ Ansatz, Eq. (5)
- 8: $\mathcal{L}_{\text{PDE}} \leftarrow$ mean-squared PDE residual of $\hat{P}$ (Eq. (4))
- 9: $\mathcal{L}_{\text{OTM}} \leftarrow \|\hat{P}(\tau^{\text{bc}}, x_{\text{lo}}, u^{\text{bc}})\|^2$, $\mathcal{L}_{\text{ITM}} \leftarrow \|\hat{P}(\tau^{\text{bc}}, x_{\text{hi}}, u^{\text{bc}}) - (e^{x_{\text{hi}}} - Ke^{-r\tau^{\text{bc}}})\|^2$
- 10: $\mathcal{L} \leftarrow \sum_k [e^{-s_k} \mathcal{L}_k + s_k]$ ▷ Adaptive weighting, Eq. (7)
- 11: Backpropagate $\mathcal{L}$; clip gradients ($\ell_2 \leq 1.0$); Adam step on (N_θ, s) ; cosine LR anneal
- 12: **end for**

Fourier forward model. A semi-analytic pricer that solves the fractional Riccati equation for the rough Heston characteristic function [5] and inverts by Fourier integration [20]. It never uses the lift, so it is the ground truth of the *true, non-lifted model* ($n \rightarrow \infty$); in the flat-volatility limit it reproduces Black–Scholes to 3.7×10^{-7} in price.

Lifted Monte Carlo. An exponential-integrator Euler scheme that integrates the n -factor lifted SDE with the $-x_i U^i$ drift handled exactly:

$$U_{t+\Delta t}^i = e^{-x_i \Delta t} U_t^i + \int_t^{t+\Delta t} e^{-x_i(t+\Delta t-s)} \sqrt{V_s} dW_s^V. \quad (8)$$

The exponential-integrator is essential: the mean reversions (x_i) span roughly ten orders of magnitude for small H , and a naive explicit Euler scheme applied to the factor equations diverges. The lifted MC matches Black–Scholes within Monte Carlo error in the flat-variance degeneracy ($\nu=\lambda=0$; 100,000 paths, 200 steps) and agrees with the Fourier forward model for large n . At finite n , the lifted MC is the *exact solution of the forward PDE the*

network is asked to solve; the Fourier forward model is the exact solution of the true model.

Precision of the reference. Every solve-error in this paper is measured against this simulator, so its own error must be smaller than the quantity measured (Table 2). All validation runs use 10^6 antithetic paths and 1 600 time steps with a terminal-forward control variate. The statistical standard error, computed on the antithetic *pair means* (the independent-sample formula overstates it by 14–31 per cent), is 0.69–0.81 px bp against solve-errors of 15–25 px bp. The time-discretisation bias, which no amount of sampling removes, is measured by a common-random-numbers refinement at 0.2–1.3 px bp. The step count matters: at the stiff factor dynamics of the calibration of Section 7.4, 400 steps leave 3.2–3.8 px bp of bias at $n=16$ and 32, comparable to the lift-error itself.

Table 2: Precision of the lifted Monte Carlo reference at the calibration of Section 7.4, $T=0.15$, in price basis points. Standard errors are RMS over strikes; the naive column is the independent-sample formula, which is not valid under antithetic sampling and is shown only to quantify the overstatement. Time-discretisation bias is from a common-random-numbers refinement against 3 200 steps.

n	SE, 2×10^5	naive SE	SE, 10^6	bias, 400	bias, 1 600
4	1.53	2.00	0.69	0.56	0.21
8	1.67	2.02	0.75	0.65	0.17
16	1.76	2.04	0.79	3.21	0.82
32	1.80	2.05	0.81	3.81	1.27

5.2. Error decomposition

Rather than reporting a single “network-versus-true-model” figure, we decompose the error per strike and in price space (where the signed contributions add exactly),

$$\underbrace{(\text{PINN} - \text{Fourier})}_{\text{total forward error}} = \underbrace{(\text{PINN} - \text{MC})}_{\text{network solve-error}} + \underbrace{(\text{MC} - \text{Fourier})}_{\text{lift (model) error}}. \quad (9)$$

The *solve-error* measures how accurately the network solves the forward PDE it was given; it is the quantity training controls, and it is the AI method’s own contribution to inverse-problem error. The *lift-error* is a forward-model

approximation that only increasing n can reduce; it is not the network’s responsibility, and no amount of network re-training removes it. Conflating the two gives a systematically pessimistic view of the AI method and leads to the wrong engineering response inside a calibration pipeline—e.g. training longer when the correct response is to raise n .

Units, stated once. Price errors are in basis points of spot (px bp; $10^{-4} \times S$) wherever the decomposition (9) is used, because it is additive in price and not in implied volatility; implied-volatility errors are in vol bp (10^{-4} in volatility), the unit a practitioner reads. At the money, $T=0.15$ and $S=1$, the Black–Scholes vega is 0.154, so

$$100 \text{ vol bp} \approx 15.4 \text{ px bp} \quad (\text{at the money, } T=0.15).$$

The conversion is pointwise: vega falls away in the wings, so an RMSE over the strike range 0.85–1.15 converts at roughly 12–14 px bp per 100 vol bp. The two components of (9) carry opposite signs, so their RMSEs do not add even though the signed errors do.

5.3. Homogeneity re-scaling

We train the network at fixed strike $K=1$ and read off the smile $C(1, K)$ through the call’s degree-one homogeneity:

$$C(1, K) = K C(1/K, 1), \tag{10}$$

which holds exactly because the variance dynamics are independent of the spot level; omitting it compares mismatched options and inflates the apparent error by an order of magnitude.

6. The calibration inverse problem: data and methods

6.1. Data

We use a single BTC option-chain snapshot from Deribit, a cryptocurrency derivatives exchange, retrieved through its public API (no API key required) on **3 July 2026 at 05:57 UTC**, at a spot of \$62,191. The raw snapshot holds 912 quoted instruments across 13 expiries.

Because the referees asked for the data handling to be stated precisely, we give the filter chain in full. From the raw chain we (i) take the mid price $\frac{1}{2}(\text{bid} + \text{ask})$ where a two-sided quote exists and otherwise fall back to

Deribit’s *mark* price; (ii) keep only the out-of-the-money leg at each strike (calls for $K \geq F$, puts for $K < F$), which is the more liquid and tighter-quoted side; (iii) drop $|k| > 0.30$ with $k = \log(K/F)$; (iv) drop quotes below one currency unit, where the implied vol is dominated by tick noise; and (v) recompute the implied volatility from the mid by Brent inversion, dropping any point falling outside $[0.01, 2.0]$. This leaves 286 points, from which we retain at most eight per maturity, spread evenly in log-moneyness so that each smile constrains shape and not merely level. The fitted surface therefore comprises **104 points across 13 maturities** (T from 0.002 to 0.98 years) spanning a moneyness range $K/F \in [0.77, 1.35]$; of these, 97 return a finite model implied volatility at the calibrated parameters and enter the reported RMSE.

Two disclosures about that surface. First, **11 of the 104 fitted points carry no two-sided quote** and therefore enter through Deribit’s mark price, which is an exchange model value rather than a tradable one. Second, **12 of the 104 had no traded volume** in the snapshot window. We do not filter on volume or open interest: Deribit’s book-summary endpoint does not return open interest, and imposing a volume filter would remove most of the long maturities entirely. Section 7.4 reports the fit restricted to the liquid moneyness core as a robustness check, and Section 6.2 reports static-arbitrage diagnostics on the surface that survives the filters above.

Two further datasets, used only for robustness. The checks of Section 7.5 use two further datasets, both collected by the authors through the same public API. (i) Five-minute bars of the Deribit BTC perpetual, August 2018 to August 2026 (841k bars), used only for the realised-volatility estimate of H . (ii) A daily implied-volatility panel built from the Deribit trade tape (41.5M option trades). Its entries are completed trades, not quotes, so the two-sided-quote filter cannot be applied; we restrict it to $|k| \leq 0.35$, $T \in [0.02, 1.0]$ and implied volatilities in $[0.01, 2.0]$, retain at most eight points per maturity as above, and keep Wednesdays with at least four maturities and 25 points. A comparison across weeks of this panel is sound, but it is not a like-for-like extension of the quote-based headline snapshot. Both datasets accompany the code in derived form.

We set $r=0$, the standard convention for crypto, where there is no clean carry, and take the forward to be $F = S e^{rT} = S$ with S the underlying index price Deribit reports alongside each instrument. Implied volatilities are computed from mid prices with a robust Brent Black–Scholes inverter. The

inverse-problem objective throughout is the vega-weighted implied-volatility RMSE between market quotes and the forward operator’s output,

$$\mathcal{L}(\Theta) = \sqrt{\frac{\sum_j w_j [\sigma_j^{\text{mod}}(\Theta) - \sigma_j^{\text{mkt}}]^2}{\sum_j w_j}}, \quad w_j = \max\left(S \phi(d_{1,j}) \sqrt{T_j}, 10^{-6}\right), \quad (11)$$

with ϕ the standard normal density and $d_{1,j}$ evaluated at the market implied volatility, so the weights are fixed across the optimisation. Vega weighting lets well-determined near-the-money points dominate, which is standard practice. We flag one asymmetry for the reader: the *objective* is vega-weighted as in (11), whereas the headline fit quality we report in Section 7.4 is the *unweighted* RMSE over the same points, which is the more conservative of the two.

6.2. Static-arbitrage diagnostics

Attributing a coarse fit to the forward model’s expressiveness, as we do in Section 7.4, is only legitimate if the target surface is itself arbitrage-consistent: quotes carrying butterfly or calendar violations cannot be fitted by *any* arbitrage-free model, and the residual would then be the data’s doing rather than rough Heston’s. Crypto surfaces are wide and thinly quoted in the wings, so this is a real possibility and we test it rather than assume it.

On undiscounted calls with $r=0$, putting the out-of-the-money puts on a common footing through parity, we check three conditions on the filtered surface: convexity in strike at each maturity (butterfly), the slope bound $-1 \leq \partial C / \partial K \leq 0$ (vertical spread), and monotonicity of total implied variance $w(k, T) = \sigma^2 T$ in T at fixed log-moneyness (calendar). The result is clean: **no violations** in 78 butterfly triples, 91 vertical pairs or 300 calendar comparisons. The margins are however slim — the smallest butterfly margin is 0.28 px bp and the smallest calendar increment is 8.2×10^{-5} in total-variance units — so the surface is arbitrage-free but only barely, which is itself a fair description of a wide crypto smile. The calibrated model’s own surface passes the same tests with larger margins (0.98 px bp butterfly), as an arbitrage-free model must.

Separately, put–call parity holds on the raw chain: of 383 strikes quoted two-sided on both legs, **none** violates $C - P = F - K$ by more than the quoted bid–ask spread, though the residual RMS of 46 px bp (maximum 163) is a reminder of how wide those spreads are.

The conclusion we draw is narrow but useful: the coarseness of the fit reported in Section 7.4 *cannot* be blamed on static-arbitrage violations in the target, because there are none. It is a statement about the forward model.

The liquid core. Is the coarse fit driven by the wings? We refit on the liquid core from both ends of the H ridge of Section 7.5 (Supplementary Table S2). On the fifty-three points with $|k| \leq 0.10$ the full-surface fit scores 160 vol bp weighted and 511 unweighted, and refitting on those points alone improves this only to 146 and 450: a model that stays above 450 vol bp on the most liquid quotes is not being defeated by illiquid ones. The recovered H , however, depends strongly on the window—0.020 on $|k| \leq 0.10$, 0.057 on $|k| \leq 0.20$, 0.172 on the full surface—and on both restricted windows the two starts converge to the same parameters. The non-identification of Section 7.5 is a property of the far wings.

6.3. Three routes to the inverse problem

We compare three ways of solving the same calibration inverse problem.

1. **True-forward-model loop.** Differential evolution (60 generations, all 5 parameters) with the Fourier forward model inside the objective, followed by Nelder–Mead polish. This is our from-scratch, exact-forward-operator reference.
2. **AI-only.** Gradient descent on the model parameters using the parametric PINN as a differentiable forward operator, with zero calls to the true forward model.
3. **Hybrid.** The parametric PINN’s single-shot solution provides the initial guess; a short Nelder–Mead polish against the true forward model then sharpens it.

Optimiser settings, stated once. Two distinct budgets appear in this paper and conflating them would misstate the speed-up, so we separate them explicitly. The *full-budget market calibration* of Section 7.4 uses SciPy’s `differential_evolution` with `maxiter=60`, `popsiz=12`, `tol=10-4`, mutation (0.5, 1.0), recombination 0.7 and seed 1, the built-in polish disabled, evaluated in parallel with deferred population updating, followed by a separate Nelder–Mead refinement (`maxiter=200`, `xatol=10-4`, `fatol=10-6`) confined to the box. The box is $H \in [0.02, 0.45]$, $V_0, \theta \in [0.005, 0.60]$, $\lambda \in [0.05, 12]$, $\nu \in [0.05, 2]$, $\rho \in [-0.95, -0.05]$; every parameter of the reported fit is interior to it. The *three-route comparison* of this section

runs on the single $T=0.15$ slice with a deliberately lighter global budget—`maxiter=30`, `popsiz=10`, seed 1—and a Nelder–Mead polish capped at 120 iterations. All forward-model evaluation counts we report, including the 1736-evaluation from-scratch baseline against which the $8.8\times$ saving is measured, come from this lighter configuration; they are not comparable to the full-budget market fit, and we do not claim they are. A search box that constrains rather than contains the optimum is a quiet source of false precision—a tighter box held λ on its bound and moved the fitted H —so we check every reported fit for bound contacts and state the one that remains (Section 7.5).

7. Results

7.1. The forward operator scales where the grid cannot

Our headline forward-operator experiment trains one architecture (width 64, depth 4) at $n=4, 8, 16, 32$ on the BTC calibration of Section 7.4 at $T=0.15$, over eight independent seeds per n (Table 3, Figure 1). The solve-error against the lifted MC is 15.2 ± 4.1 , 21.2 ± 5.2 , 24.6 ± 5.0 and 18.3 ± 6.1 px bp at $n=4, 8, 16, 32$: it rises to $n=16$, falls back at $n=32$, and grows by a factor of 1.21 across the eight-fold rise in dimension. On all thirty-two seed-level points, a least-squares fit of log solve-error on $\log n$ gives a slope of 0.09 ± 0.08 , significantly below one, and a one-sided permutation test of monotone increase over 20 000 relabellings gives $p = 0.14$: we cannot reject that the solve-error is flat in n . Its absolute level reflects the stiff factor dynamics of this calibration ($\lambda = 4.94$, $\nu = 1.37$). Training cost does not grow with n either: one iteration costs 84–120 ms at every n on an otherwise idle machine, because the factor-Hessian computation is vectorised over the full double sum.

The lift-error falls from 8.8 px bp at $n=4$ to 3.8 at $n=8$ and by $n=16$ reaches the resolution floor of the reference, whose standard error and time-discretisation bias together amount to 1.8 px bp at $n=16$ and 2.7 at $n=32$ (shaded in Figure 1); we draw no conclusion from its value beyond that point. The floor is the Monte Carlo, not the Fourier price: the latter is converged to 0.1 vol bp under a sixteen-fold refinement of its time grid and frequency truncation.

Compare this against a finite-difference forward solver. A naive full-tensor grid on the same $(n+1)$ -dimensional PDE at a modest 50 nodes per dimension needs 50^{n+1} points: 3.1×10^8 at $n=4$, 2.0×10^{15} at $n=8$, and 1.2×10^{56} at

$n=32$. Even $n=8$ —roughly 16 petabytes in double precision—is out of reach as a forward operator inside any iterative inverse-problem solver, while the network trains every n on a laptop. We stress that 50^{n+1} is an illustrative memory bound for the naive full-tensor discretisation and not a benchmark against a tuned solver. Structured schemes reach considerably further: sparse grids [23, 24] break the full-tensor node count, ADI splitting handles the correlation cross-terms efficiently in the classical Heston setting [25], and low-rank tensor-train representations [26] compress high-dimensional operators where the solution admits low-rank structure. Each, however, still degrades with n —sparse-grid node counts and error constants grow super-linearly in the dimension, and the tensor-train ranks that make the method cheap are eroded by precisely the coupled ρ - ν cross-terms this PDE carries—and none is standard or validated for the lifted rough-volatility PDE. We therefore make the conservative claim, and only it: a competitive grid-based forward solver at $n=16$ – 32 is *impractical* as the engine of a calibration loop, not provably impossible. Establishing the stronger claim would require building and tuning such a solver, which is a research programme in its own right.

Table 3: Dimension scaling of the forward operator: single-point PINN at $T=0.15$ on the BTC calibration of Section 7.4. Solve-error is the eight-seed mean $\pm$ SD in price basis points of spot, against a lifted Monte Carlo of 10^6 antithetic paths at 1 600 steps; lift-errors at $n=16$ and 32 are below that reference’s resolution floor (see text). Cost is milliseconds per training iteration, measured with no other job running.

n	solve-error (px bp)	lift-error (px bp)	ms / iteration	FD grid $50^{(n+1)}$
4	15.2 ± 4.1	8.8	120	3.1×10^8
8	21.2 ± 5.2	3.8	94	2.0×10^{15}
16	24.6 ± 5.0	1.0	92	7.6×10^{28}
32	18.3 ± 6.1	3.4	84	1.2×10^{56}

7.2. Sensitivity to architecture and collocation, and where the variation lives

Is the width-64 / depth-4 choice a knife-edge, and does the variation of the solve-error across n come from the lifted PDE or from the optimiser? For the second we use the structure of a-priori PINN error estimates [27, 28]—the solve-error is bounded by a stability constant $C(n)$ times the residual—and score every trained network on a fixed, seeded set of 20 000 collocation points it never saw (Supplementary Tables S3–S5). Three findings, the first two against us. The architecture used throughout is not the best: width 128 at

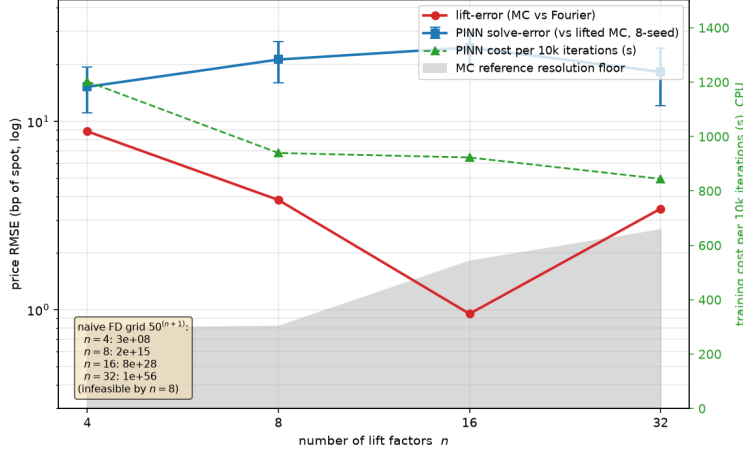


Figure 1: Dimension scaling of the forward operator. PINN solve-error (blue, eight-seed mean $\pm$ SD) shows no statistically detectable growth over an eight-fold rise in n (log-log slope 0.09 ± 0.08 ; permutation $p = 0.14$), and training cost per iteration (green) is flat. The lift-error (red) falls into the Monte Carlo reference’s resolution floor (grey: standard error combined with time-discretisation bias) by $n=16$, beyond which it is not resolvable. The inset shows the naive finite-difference grid size 50^{n+1} , infeasible by $n=8$.

depth 4 reaches 9.7 ± 0.5 px bp at $n=4$ against 18.2 ± 4.2 for our width 64, with four times the parameters; we keep width 64 for continuity. Collocation is under-resolved at high dimension: at $n=32$, doubling n_{col} to 4000 halves the solve-error (22.1 to 11.0 px bp), the quadrature term of the estimate at work. And the held-out residual is flat across n to within twelve per cent while the solve-error moves by up to fifty-seven per cent (Figure 2): the network solves the PDE equally well at every dimension, so the variation lies in the stability constant of the lifted operator, not in the optimiser. With two seeds per cell and no significant trend in the eight-seed sweep, we offer this as where the variation lives, not as evidence that the error grows with n .

7.3. Error decomposition and a design rule for n

The value of the decomposition (9) is that it tells a user which lever to pull, and the answer depends on the calibration. At the parameters of the submitted version ($H=0.09$, $\nu=0.30$) a “network versus true model” error of 121 vol bp at $n=4$ was almost entirely lift—133 vol bp against 37 of solve-error—and the lever was more factors. At the corrected calibration the split

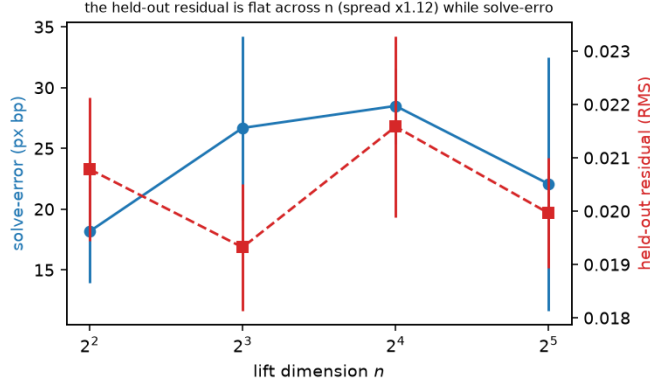


Figure 2: Solve-error (left axis) and held-out PDE residual (right axis) across the lift dimension n , width 64 / depth 4, two seeds.

reverses: in implied-vol RMS the solve-error is 112, 152, 176 and 136 vol bp at $n=4, 8, 16, 32$, against lift-errors of 76, 33, 9 and 26. The network is the binding constraint at every n , and Section 7.2 shows that lever is real: a wider network or twice the collocation roughly halves the solve-error. The two components also partly cancel at small n , which a single end-to-end figure would hide.

The design rule that holds in both regimes is *add lift factors only while the lift-error exceeds the solve-error; beyond that, improve the network*. The crossover is calibration-dependent: near $n=8-10$ at the submitted parameters, below $n=4$ at the corrected fit, whose smoother kernel is easier to approximate and whose stiffer dynamics are harder to solve. It also shifts with maturity: in a sweep at the submitted parameters the lift-error at $n=4$ grew from 8.6 to 31.8 px bp as T rose from 0.05 to 0.5.

The network’s error is not symmetric in strike (Supplementary Figure S1). It over-prices at and above the money at every n , by +138 to +221 vol bp ATM, while the put wing is smaller and near zero at $n=32$ —a watch item inside a calibration loop, since a one-signed error biases the recovered skew.

The $n=32$ bias, taken apart. Factor-tail coverage is symmetric by construction, so asymmetry is not the mechanism. Two others are separable by ablation (Table 4): a noisy forward-variance anchor, since the baseline is anchored on a pre-flight Monte Carlo of 4000 paths and a level error in the anchor is a level error in the price; and under-coverage of the factor tails in

32 dimensions. A precise anchor of 200 000 paths lowers the solve-error from 16.8 to 14.3 px bp and the ATM bias by thirty vol bp, so a noisy anchor is part of the mechanism. Widening the tails does the opposite, raising the solve-error by 86 per cent: wider tails spend collocation points where the solution rarely goes, at the expense of the bulk where the price is set. The bias that survives a precise anchor is the variation that Section 7.2 places in the stability constant.

Table 4: Ablation of the $n=32$ directional bias, three seeds, against the 1 600-step reference. Solve-error in px bp; signed implied-vol error by strike bucket in vol bp (network minus lifted MC).

Configuration	solve-error	puts	ATM	calls
baseline (J1 setting)	16.8 ± 10.1	+3	+132	+186
precise anchor	14.3 ± 5.8	−17	+102	+162
wide tails	31.3 ± 8.2	+124	+261	+315
precise anchor + wide tails	32.3 ± 9.4	+84	+268	+338
Reference: 10^6 paths, 1 600 steps; its own bias 1.27 px bp.				

7.4. Solving the inverse problem: the correlation is pinned, the roughness is not

Calibrated to the full Deribit BTC surface with a full-budget optimiser (differential evolution, 60 generations, all 13 maturities and all 104 points, followed by a bounded Nelder–Mead polish against the true Fourier forward model), rough Heston returns

$$H=0.172, \quad \rho=-0.426, \quad V_0=0.105, \quad \theta=0.245, \quad \lambda=4.94, \quad \nu=1.37,$$

at a vega-weighted implied-vol RMSE of 113 vol bp and an unweighted RMSE of 416 vol bp (Figure 3). The unweighted figure is dominated by one part of the surface. On the thirty-two points with maturities beyond two months the model fits to 52 vol bp, comfortably inside a quoted Bitcoin option spread; on the forty-eight points under two weeks it misses by 608. Rough Heston describes the BTC term structure well and fails only on the shortest-dated options, whose smiles are too steep for this model to follow. That is a statement about the forward model’s expressiveness, not about the AI forward operator, and it is orthogonal to the solve-error studied above.

A correction to the submitted fit. The submitted version reported $H=0.090$, $\rho=-0.792$, $\nu=0.300$ at 496 vol bp, from an objective with a defect we found during revision: when a model price fell at or below intrinsic value, the implied-volatility inversion failed and the point was silently dropped, so a parameter vector at which badly fitted points *failed to price* scored better than one that priced them. A fixed Fourier truncation compounded this, mispricing the shortest maturities by up to 0.33 in implied volatility; the truncation frequency must scale with $1/\sqrt{VT}$. Both are corrected: every point counts in every evaluation, a point priced at or below intrinsic is charged its full error, and a genuine pricer failure makes the vector infeasible. Under the corrected objective the submitted vector scores 204 vol bp weighted against 113 for the fit above. The point value $H=0.172$ should be read with Section 7.5, which shows the surface pins ρ but leaves H free over $[0.04, 0.30]$.

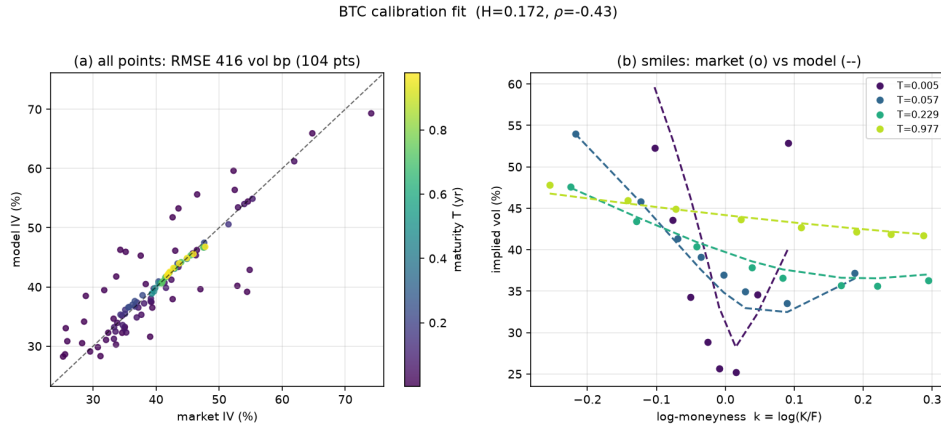


Figure 3: Rough Heston calibrated to the Deribit BTC surface (all 104 points, 13 maturities), solving the inverse problem with the true Fourier forward model. (a) Market versus model implied volatility at every point (dashed: identity; colour: maturity); the outliers are the shortest maturities. (b) Per-maturity smiles for four representative maturities: market (markers) versus model (dashed). Beyond two months the model fits to 52 vol bp; the residual is concentrated in options under two weeks and reflects a model-expressiveness limitation, not a forward-operator artefact.

7.5. Is the recovered roughness real, and is it stable?

Whether Bitcoin volatility is rough rests, so far, on one option surface read through a six-parameter inverse problem. The inverse problem might not identify H sharply, and a single snapshot says nothing about whether

the roughness is a stable feature of the market. We test both, the second with an estimator that never touches an option price.

Whether the surface identifies H . It does not. Three full-budget fits of the same surface under the same objective, differing only in the search box, return $H=0.019$, 0.096 and 0.172 at weighted RMSEs of 114.9 , 113.7 and 113.2 vol bp—within 1.7 vol bp—while ρ agrees to 0.007 . A profile likelihood makes this precise (Figure 4; values in Supplementary Table S6). Fixing H on a twelve-point grid and re-optimising the other five parameters from both ends of the ridge, the objective moves by 1.3 vol bp as H runs from 0.04 to 0.26 , and every H in $[0.04, 0.30]$ fits within five per cent of the best, with ν climbing from 0.80 to 1.94 and λ from 2.3 to 7.6 along the ridge to compensate. Profiling ρ gives the opposite picture: the five-per-cent set is a single grid point at -0.45 . *The surface identifies the correlation sharply and the roughness barely at all.* Because the whole interval lies far below $\frac{1}{2}$, the claim that Bitcoin volatility is rough survives; the point estimate does not, and we do not report one. Two caveats. Each profile point is a local re-optimisation, so the values are upper bounds and the interval, if anything, too narrow; and from $H=0.30$ upward the re-optimised ν sits on its cap of 2 , so the upper end of the interval is a lower bound on the true one. We run every PINN experiment at the best fit, a deliberately demanding choice since its stiff dynamics make every solve-error larger.

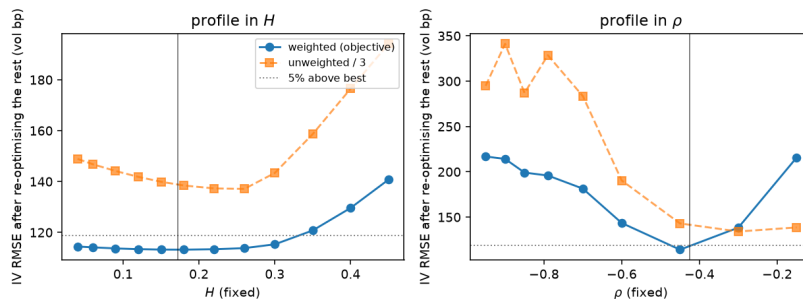


Figure 4: Profile likelihood on the headline surface. Left: H fixed on a grid, the other five parameters re-optimised from both ends of the ridge; right: the same for ρ . Vertical line: the reference fit; dotted line: 5% above the best profile value, the interval rule.

What the ridge costs in risk terms. Non-identification matters only if the quantities a desk uses depend on where along the ridge one sits. Pricing a

three-month variance-swap fair strike, a 25-delta risk reversal and a 90/110 collar with the exact forward model for all nine ridge vectors (Supplementary Table S7), the variance swap moves by 0.60 vol points on a level of 43.4, the risk reversal by 0.46 vol points, and the collar by 5.7 px bp. The ambiguity in H is close to harmless for these instruments, because ν and λ compensate and they depend on the compensated combination. By contrast, the hybrid vectors of Section 7.6, fitted to one maturity slice, price the risk reversal half a vol point steeper: which part of the surface a calibration is asked to fit matters more than where on the ridge it lands.

An estimate that does not use the option surface. Following Gatheral, Jaisson and Rosenbaum [1], we estimate H from realised volatility. From five-minute log returns of the Deribit BTC perpetual we build daily realised variance, set $\sigma_t = \sqrt{RV_t}$, and form $m(q, \Delta) = \mathbb{E}|\log \sigma_{t+\Delta} - \log \sigma_t|^q$. For a fractional process $m(q, \Delta) \sim \Delta^{qH}$, so regressing $\log m$ on $\log \Delta$ for each q gives slopes $\zeta(q)$, and regressing ζ on q through the origin gives H . Over 2921 days (841k bars) this returns

$$H_{\mathbb{P}} = 0.111, \quad 95\% \text{ CI } [0.102, 0.150],$$

from a 60-day moving-block bootstrap; the regression’s analytic standard error ignores the persistence of log-volatility and is an order of magnitude too small. The estimate uses no option data, no lift, no pricer and no optimiser, and it lands inside the option-implied interval: where the surface cannot choose a point on its ridge, this estimate can, and it chooses one well inside the rough regime. It agrees with Takaishi [29] over an earlier, disjoint sample.

Stability across the spot-ETF approval. We split the realised-volatility series at the approval of US spot Bitcoin ETFs on 11 January 2024—calendar 2023 against February–December 2024, January excluded—and re-estimate:

$$H_{\mathbb{P}}^{\text{pre}} = 0.064 \text{ [0.018, 0.098]}, \quad H_{\mathbb{P}}^{\text{post}} = 0.062 \text{ [0.022, 0.079]},$$

a difference of +0.002 against a combined bootstrap standard error of 0.025. The estimator is biased low on short windows, so these are comparable with each other but not with the full-sample figure.

The same study on option surfaces uses the trade-based panel of Section 6.1: 52 pre-ETF and 47 post-ETF weekly surfaces, each fitted from both ends of the ridge and never from the previous week (Table 5; weekly series in

Supplementary Figure S2). From either start, mean weekly H is statistically indistinguishable across the split, in agreement with the physical-measure result. What does change is the vol-of-vol, which falls from 1.96 to 1.63, and the fit, which degrades from 124 to 151 volbp; both intervals exclude zero. The weekly ρ sits near zero against -0.43 on the headline snapshot; that gap is the trade-versus-quote provenance difference, not the market, so we compare the panel across weeks and do not interpret its level.

Table 5: Option-implied parameters before and after the spot-ETF approval, 52 and 47 weekly surfaces. Mean $\pm$ SD per regime; difference pre–post with a 4-week block-bootstrap 95% interval and Welch p .

Quantity	pre-ETF	post-ETF	diff.	95% CI	p
H (canon. start)	0.099 ± 0.094	0.107 ± 0.133	-0.008	$[-0.071, +0.059]$	0.723
H (wide start)	0.163 ± 0.144	0.132 ± 0.154	$+0.031$	$[-0.059, +0.092]$	0.301
ρ	0.004 ± 0.167	0.086 ± 0.215	-0.083	$[-0.167, -0.003]$	0.037
ν	1.961 ± 0.570	1.631 ± 0.724	$+0.330$	$[+0.162, +0.689]$	0.014
fit (vol bp)	124.1 ± 47.3	150.6 ± 45.0	-26.5	$[-50.9, -15.8]$	0.005
basin gap (vol bp)	46.9 ± 83.8	43.4 ± 47.7	$+3.5$	$[-20.8, +33.6]$	0.797
Weeks preferring the smoother- H basin: 35% pre, 23% post; $n = 52$ and 47 weeks.					

7.6. A parametric AI forward operator inside the inverse problem

A single network cannot cover both the breadth of a parameter box and the accuracy of a point-trained forward operator. The parametric network adds $(H, \nu, \rho, \lambda, \theta)$ as five inputs, holding V_0 and the maturity fixed, with $n=10$ (the crossover rule of Section 7.3), width 96 and depth 5. We train it on two boxes with identical architecture, budget and seeds—the original box ($\nu \leq 0.6$, $\theta \leq 0.35$) and a widened one ($\nu \leq 1.0$, $\theta \leq 0.45$) that contains the optimum of the $T=0.15$ slice—so the question of whether the box was mis-specified becomes a controlled ablation.

Solve-error depends strongly on where in parameter space it is measured: 3.6 to 9.1 px bp at mild probe vectors, 32–38 at the rough, low- H probe. At the calibrated vector, which has $\nu = 1.37$ and lies outside *both* boxes, the widened box gives 9.1 ± 3.4 px bp against 33.8 ± 2.5 : both are extrapolations, and the widened box extrapolates a shorter distance. The error concentrates where H is small and ν large (Figure 5), and it is an interaction rather than the sum of two effects: adding a ν/H term to an additive regression raises R^2 from 0.42–0.49 to 0.67–0.68, a partial R^2 of 0.19 to 0.25.

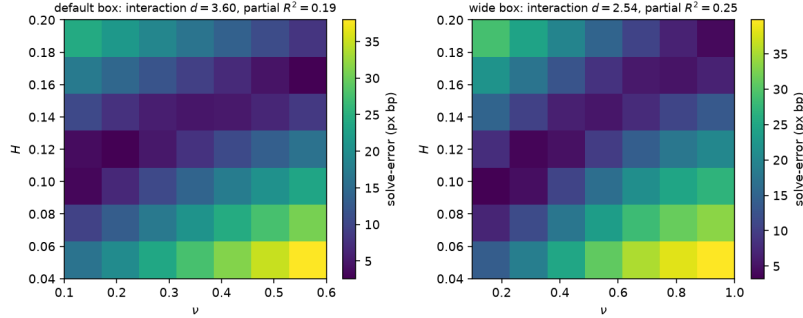


Figure 5: Parametric solve-error over (H, ν) at the box centre in (ρ, λ, θ) , $n=10$, seed-averaged, against a lifted Monte Carlo at every cell. Titles give the interaction coefficient d of $\text{err} \sim a + b/H + c\nu + d\nu/H$ and its partial R^2 .

Used alone, the parametric operator is fast (~ 2 s, zero true-model calls) but poor (Table 6, Figure 6): 161 ± 15 vol bp on the original box and 181 ± 97 on the widened one, whose seeds reach 316, 92 and 134. The submitted version attributed this to the slice optimum lying outside the box; the ablation does not support that. Containing the optimum is necessary but not sufficient—the network fits its own imperfect forward map, and over a wider box that map is less accurate where it matters—and we do not recommend the standalone operator on either box. As a warm start for a Nelder–Mead polish against the true forward model it is effective on both: 23.8 and 23.6 vol bp against 22.7 from scratch, on a slice whose market implied vols span roughly 34 to 52 per cent, in 197 and 205 true-model evaluations against 1 736—a saving of $8.8\times$ ($8.7\text{--}8.9\times$ across seeds) and $8.5\times$ —and 17 s against 142 s. The hybrid does not need the training box to be chosen well.

The saving is amortised, and it has a concrete break-even. Each warm-started recalibration saves about 1 535 true-model evaluations, or 125 s; training the parametric network costs 87–110 minutes. It pays for itself after 42 to 53 recalibrations of the same maturity slice—about two months of daily recalibration—and below that the exact forward model from scratch is the cheaper tool. The training times were measured on a shared machine, which inflates them, so the break-even is if anything overstated.

7.7. Against the forward operators practitioners use

A reader wanting a fast forward operator for rough Heston would not reach for a PINN first. They would price with the Fourier transform, run

Table 6: Solving the calibration inverse problem for the BTC $T \approx 0.15$ slice: the true-forward-model loop from scratch, and the AI-only and hybrid routes with the parametric network trained on the original box and on a widened box that contains the slice optimum. Three network seeds each; fit quality is judged by the true Fourier forward model over the same feasible region.

Method	Fit RMSE (vol bp)	Wall-clock (s)	True-model evals
True-forward-model loop, from scratch	22.7	142	1736
AI-only (single-shot), original box	161 ± 15	~ 2	0
Hybrid (AI warm start, true-model polish), original box	23.8 ± 0.0	17	197 ± 1
AI-only (single-shot), widened box	181 ± 97	~ 2	0
Hybrid (AI warm start, true-model polish), widened box	23.6 ± 0.1	17	205 ± 1

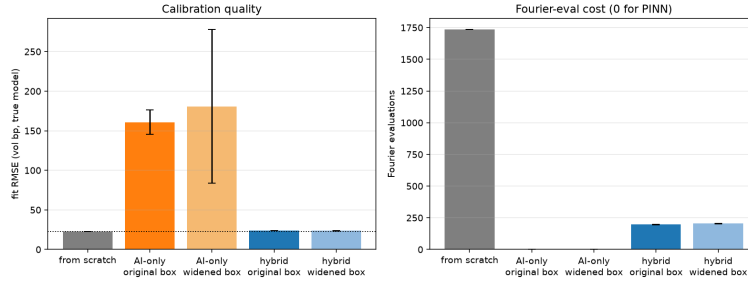


Figure 6: Hybrid solution of the inverse problem: fit quality (left) and true-forward-model evaluations (right). The hybrid matches from-scratch quality at $8.5\text{--}8.8\times$ fewer true-model evaluations whichever box the network was trained on; the AI-only route is fast but inaccurate, and widening the box to contain the optimum makes it no better on average and far less reliable.

a short lifted Monte Carlo, or fit a network to exact-pricer labels in the manner of Bayer et al. [13] and Horvath et al. [14]—the method class of the calibration networks of Liu et al. [30]. Table 7 compares all three with our operator. The supervised surrogate matches the parametric network’s protocol exactly: the same five inputs, the same 21-point log-moneyness grid at $T=0.15$, the same widened box, split by parameter vector so no strike of a test vector is seen in training. The comparison goes against us on the axes most readers check first. Trained on 64 353 Fourier-labelled vectors, the surrogate reaches 5.7 ± 0.1 vol bp against the transform method, and 9.3 ± 1.0 in the rough, high- ν corner; it is more accurate than the PINN against Fourier and as fast once trained, and on one-shot cost the Fourier pricer and a short lifted Monte Carlo beat a PINN training run by orders of magnitude. What the surrogate cannot do is exist without an exact pricer—every label came from the fractional Riccati solver—whereas the PINN needs only the PDE, so it transfers to models with no transform method; and the PINN yields the solve/lift decomposition of Section 5.2, which a network fitted to Fourier labels cannot separate. Those two properties are the case for this operator, and we claim nothing beyond them.

8. Discussion

Convergence of the AI forward operator. The finding we care about most is that the network’s error in solving the lifted forward PDE does not measurably degrade as the dimension grows—a log-log slope of 0.09 ± 0.08 over eight seeds, with no rejection of flatness—and that its cost per iteration is flat, while a naive grid becomes impractical as an inverse-problem engine. Structured grid methods go further than the naive count, so the defensible claim is *impractical, not impossible*; the contrast is nonetheless a concrete instance of the mesh-free advantage inside an inverse-problem pipeline, and it holds at a deliberately stiff calibration.

Four design choices are load-bearing. Ablations at the parameters of the submitted version showed each stabiliser to matter. Without the hard-constrained ansatz the soft initial-condition penalty oscillates against the residual and training fails to converge; without the integrated-variance anchor a uniform level bias of about 120 vol bp remains; without MC-informed factor sampling the smile flattens and the error roughly doubles; without self-adaptive weighting and the τ -curriculum the seed variance is markedly higher. Section 7.3 adds a fifth: the precision of the anchor itself.

Table 7: Forward operators for the lifted rough-Heston smile at $T=0.15$. Accuracies are not comparable across rows, and the column says against what each is measured: the surrogate is trained on Fourier labels and scored against them, in implied-vol basis points, while the PINNs are scored in price basis points against an independent lifted Monte Carlo at the same n and carry the lift-error against Fourier on top. The parametric range runs from mild parameter vectors to the rough, high- ν corner. Both PINNs and the surrogate are differentiable in their inputs; the Fourier pricer and the Monte Carlo are not. Costs for the PINNs are CPU wall-clock; the parametric figure was measured on a shared machine and is indicative.

Operator	Offline cost	Exact pricer offline?	Accuracy (against)	Solve/lift split?
Fourier (fractional Riccati)	none	is one	exact by construction	no
Lifted MC, 10^6 paths	none	no	reference; SE 0.7–0.8 px bp	no
Single-point PINN, $n=4$	≈ 20 min per vector	no	15.2 px bp (lifted MC)	yes
Parametric PINN, $n=10$	80–125 min, once	no	3.6–38 px bp (lifted MC)	yes
Supervised surrogate (MLP)	71 min labels + 26 s	yes	5.7 vol bp (Fourier); 9.3 in the rough corner	no

Validating an AI forward operator. A learned operator that replaces an exact solver inside an inverse problem is easily validated against the wrong reference. Our network solves the finite- n lifted PDE, whose exact solution is the lifted Monte Carlo, not the true model; comparing it straight to the true model conflates network error with model error. The decomposition showed the two carry opposite signs and that which one dominates depends on the calibration—the lift at the submitted parameters, the network at the corrected ones—so a single end-to-end figure would have pointed at the wrong lever in one case or the other. *Validate against a same-fidelity reference, report the two errors separately, and let their crossover set the approximation order.* None of this is special to rough volatility.

What the inverse problem can and cannot identify. The calibration surface pins the correlation and bounds the Hurst exponent only to an interval, along a ridge on which the vol-of-vol compensates. That is a property of the inverse problem, not of the forward operator, and it has two practical consequences:

a point estimate of H from option prices alone should not be reported, and an independent estimator that bypasses the inverse problem, such as realised volatility, is worth more than a tighter calibration. The parametric operator adds a limitation of its own: used alone it fits its imperfect forward map rather than the market, and a training box that contains the optimum does not repair that. Keeping the exact forward model as the final arbiter, as the hybrid does, does.

Limitations. The headline calibration rests on one quote-based snapshot; the ninety-nine weekly surfaces used for the regime study are trade-based and serve only as a robustness check. The parametric and calibration studies fix the maturity to a single slice, and all timings are single-machine CPU figures. The network over-prices at and above the money, which would bias a recovered skew. Our architecture is not the best of those we tested, the lift-error is not resolved beyond $n=16$ by our reference, and the maturity and Hurst robustness sweep predates the corrected calibration.

Future work. Natural extensions are maturity as a network input, so one operator serves the whole surface; wider networks and more collocation at high n , which Section 7.2 shows would roughly halve the solve-error; other rough models and path-dependent payoffs; a correction for the directional bias; and a formal identifiability analysis of the calibration inverse problem as $n \rightarrow \infty$.

9. Conclusion

We have characterised a physics-informed neural network as a forward operator for the rough-volatility calibration inverse problem, on the $(n+1)$ -dimensional Markovian-lifted PDE that makes the forward problem well-posed in the first place. Across an eight-fold rise in the lifted dimension the network’s solve-error shows no statistically detectable growth and its training cost per iteration is flat, while naive finite-difference forward solvers become infeasible by $n=8$; we identify the four training choices that make this behaviour hold, and show that the held-out PDE residual is equally flat, so what variation remains lies in the stability constant of the lifted operator rather than in the optimiser. Separating solve-error from lift-error is essential to correctly attribute inverse-problem error: it told us to add factors at one calibration and to improve the network at another, and it yields a

design rule for the factor count that holds in both. Applied to live Bitcoin options, the surface pins the correlation but bounds the Hurst exponent only to $[0.04, 0.30]$ —rough, but not a point—and that ambiguity moves the variance-swap strike and the risk reversal by well under a vol point. A parametric extension of the forward operator, while not accurate enough to solve the inverse problem independently, is an effective warm start that solves it at 8.8 times fewer expensive forward-model evaluations. The methodological message is that an AI forward operator earns its place inside an inverse-problem pipeline only once its own error is separated from the model error it is approximating—and that, done this way, it degrades gracefully where grid methods cannot follow.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Supplementary material

Supplementary material associated with this article contains the compute environment (Table S1), the liquid-core refit (Table S2), the hyperparameter and collocation study (Tables S3–S5), the signed solve-error by strike (Figure S1), the profile-likelihood values (Table S6), the risk functionals for every ridge vector (Table S7) and the weekly option-implied parameter series (Figure S2).

Data availability statement

The BTC option data were retrieved from the Deribit public REST API (no API key required) at <https://www.deribit.com/api/v2>. The raw data snapshot used in the experiments, the calibrated parameter files, the two robustness datasets of Section 6.1 in derived form (weekly implied-vol

surfaces and daily realised variance), every result file the tables are generated from, and the complete PyTorch implementation are available at <https://github.com/aviral9876/physics-informed-nn-rough-vol>.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this manuscript the authors used Claude (Anthropic) to assist with prose drafting and structural organisation. After using this tool, the authors reviewed and edited all content as needed and take full responsibility for the accuracy and integrity of the published work. The mathematical derivations, experimental results, code, and all scientific claims are the authors' own.

Appendix A. Derivation of the lifted pricing PDE

We derive (4) from the lifted dynamics (3), since a referee correctly observed that the correspondence is not self-evident — in particular the presence of a *full* (i, j) double sum in the factor block, where a reader expecting independent Ornstein–Uhlenbeck factors would predict a diagonal one.

Collect the state as $Z_t = (X_t, U_t^1, \dots, U_t^n)$ with $X_t = \log S_t$. Under the risk-neutral measure the lifted system is

$$dX_t = \left(r - \frac{1}{2}V_t\right) dt + \sqrt{V_t^+} dB_t, \quad (\text{A.1})$$

$$dU_t^i = [-x_i U_t^i + \lambda(\theta - V_t)] dt + \nu \sqrt{V_t^+} dW_t^V, \quad i = 1, \dots, n, \quad (\text{A.2})$$

with $V_t = V_0 + \sum_i c_i U_t^i$ and $d\langle B, W^V \rangle_t = \rho dt$. This is a $(n+1)$ -dimensional Itô diffusion, hence Markovian — which is the entire purpose of the lift, the original Volterra variance process being neither.

The instantaneous covariation matrix of Z follows from the fact that the system is driven by only *two* Brownian motions, B and W^V , whatever the value of n :

$$\begin{aligned} \frac{d\langle X, X \rangle_t}{dt} &= V_t^+, & \frac{d\langle X, U^i \rangle_t}{dt} &= \rho \nu V_t^+, \\ \frac{d\langle U^i, U^j \rangle_t}{dt} &= \nu^2 V_t^+ & \text{for all } i, j. \end{aligned} \quad (\text{A.3})$$

The last identity is the crux. Every factor is driven by the same increment dW_t^V and differs only through its mean-reversion speed x_i , so the factor block of the covariation matrix is $\nu^2 V_t^+ \mathbf{1}\mathbf{1}^\top$ — rank one, with every entry equal, rather than diagonal. The factors are perfectly instantaneously correlated; they decorrelate only through their differing decay rates over finite time. Consequently the second-order factor term in the generator carries the unrestricted double sum $\sum_{i,j=1}^n P_{u_i u_j}$ that appears in (4), and not $\sum_i P_{u_i u_i}$.

The infinitesimal generator of Z is therefore

$$\begin{aligned} \mathcal{A}P = & \left(r - \frac{1}{2}V\right) P_x + \sum_{i=1}^n [-x_i u_i + \lambda(\theta - V)] P_{u_i} \\ & + \frac{1}{2}V P_{xx} + \frac{1}{2}\nu^2 V \sum_{i,j=1}^n P_{u_i u_j} + \rho\nu V \sum_{i=1}^n P_{xu_i}, \end{aligned} \quad (\text{A.4})$$

where we write V for V^+ to lighten notation, the truncation being active only on a set the scheme controls numerically. For a European claim with payoff $g(X_T)$, the discounted price $P(t, x, u) = \mathbb{E}[e^{-r(T-t)} g(X_T) \mid Z_t = (x, u)]$ satisfies the Feynman–Kac equation $P_t + \mathcal{A}P - rP = 0$ with $P(T, x, u) = g(x)$, which on substituting $\mathcal{A}$ and $g(x) = \max(e^x - K, 0)$ is exactly (4).

Three remarks bear on the numerics. First, the drift source $\lambda(\theta - V)$ and the diffusion coefficient $\nu\sqrt{V}$ are *shared* across factors; only $-x_i u_i$ distinguishes them, which is why the factors can be advanced with a single common Brownian draw. Second, the (x_i) span roughly ten orders of magnitude at small H , making the factor block stiff and forcing the exponential-integrator treatment of Section 5. Third, the rank-one structure of the factor covariation is what couples ρ and ν across the whole factor block, and is the reason the low-rank tensor structure that a tensor-train solver would need is eroded rather than preserved as n grows.

References

- [1] J. Gatheral, T. Jaisson, M. Rosenbaum, Volatility is rough, *Quantitative Finance* 18 (6) (2018) 933–949. doi:10.1080/14697688.2017.1393551.
- [2] M. Bennedsen, A. Lunde, M. S. Pakkanen, Decoupling the short- and long-term behavior of stochastic volatility, *Journal of Financial Econometrics* 20 (5) (2022) 961–1006. doi:10.1093/jjfinec/nbaa049.

- [3] S. L. Heston, A closed-form solution for options with stochastic volatility with applications to bond and currency options, *Review of Financial Studies* 6 (2) (1993) 327–343. doi:10.1093/rfs/6.2.327.
- [4] F. Comte, E. Renault, Long memory in continuous-time stochastic volatility models, *Mathematical Finance* 8 (4) (1998) 291–323. doi:10.1111/1467-9965.00057.
- [5] O. El Euch, M. Rosenbaum, The characteristic function of rough Heston models, *Mathematical Finance* 29 (1) (2019) 3–38. doi:10.1111/mafi.12173.
- [6] E. Abi Jaber, O. El Euch, Multifactor approximation of rough volatility models, *SIAM Journal on Financial Mathematics* 10 (2) (2019) 309–349. doi:10.1137/18M1170236.
- [7] E. Abi Jaber, Lifting the Heston model, *Quantitative Finance* 19 (12) (2019) 1995–2013. doi:10.1080/14697688.2019.1615113.
- [8] M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, *Journal of Computational Physics* 378 (2019) 686–707. doi:10.1016/j.jcp.2018.10.045.
- [9] J. Sirignano, K. Spiliopoulos, DGM: A deep learning algorithm for solving partial differential equations, *Journal of Computational Physics* 375 (2018) 1339–1364. doi:10.1016/j.jcp.2018.08.029.
- [10] J. Han, A. Jentzen, W. E, Solving high-dimensional partial differential equations using deep learning, *Proceedings of the National Academy of Sciences* 115 (34) (2018) 8505–8510. doi:10.1073/pnas.1718942115.
- [11] A. Papapantoleon, J. Rou, A time-stepping deep gradient flow method for option pricing in (rough) diffusion models, *Quantitative Finance* 25 (12) (2025) 2009–2020. doi:10.1080/14697688.2025.2572318.
- [12] A. Jacquier, v. Zurič, Random neural networks for rough volatility, preprint, arXiv:2305.01035 (2023). arXiv:2305.01035.

- [13] C. Bayer, B. Horvath, A. Muguruza, B. Stemper, M. Tomas, On deep calibration of (rough) stochastic volatility models, preprint, arXiv:1908.08806 (2019). [arXiv:1908.08806](#).
- [14] B. Horvath, A. Muguruza, M. Tomas, Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models, *Quantitative Finance* 21 (1) (2021) 11–27. [doi:10.1080/14697688.2020.1817974](#).
- [15] A. S. Krishnapriyan, A. Gholami, S. Zhe, R. M. Kirby, M. W. Mahoney, Characterizing possible failure modes in physics-informed neural networks, *Advances in Neural Information Processing Systems (NeurIPS)* 34 (2021) 26548–26560.
- [16] S. Wang, X. Yu, P. Perdikaris, When and why PINNs fail to train: a neural tangent kernel perspective, *Journal of Computational Physics* 449 (2022) 110768. [doi:10.1016/j.jcp.2021.110768](#).
- [17] A. Kendall, Y. Gal, R. Cipolla, Multi-task learning using uncertainty to weigh losses for scene geometry and semantics, *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2018) 7482–7491 [doi:10.1109/CVPR.2018.00781](#).
- [18] L. D. McClenny, U. M. Braga-Neto, Self-adaptive physics-informed neural networks, *Journal of Computational Physics* 474 (2023) 111722. [doi:10.1016/j.jcp.2022.111722](#).
- [19] S. Wang, S. Sankaran, P. Perdikaris, Respecting causality for training physics-informed neural networks, *Computer Methods in Applied Mechanics and Engineering* 421 (2024) 116813. [doi:10.1016/j.cma.2024.116813](#).
- [20] P. Carr, D. B. Madan, Option valuation using the fast Fourier transform, *Journal of Computational Finance* 2 (4) (1999) 61–73. [doi:10.21314/JCF.1999.043](#).
- [21] X. Glorot, Y. Bengio, Understanding the difficulty of training deep feed-forward neural networks, in: *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010, pp. 249–256.

- [22] D. P. Kingma, J. Ba, Adam: a method for stochastic optimization, International Conference on Learning Representations (ICLR)ArXiv:1412.6980 (2015).
- [23] H.-J. Bungartz, M. Griebel, Sparse grids, *Acta Numerica* 13 (2004) 147–269.
- [24] C. Reisinger, G. Wittum, Efficient hierarchical approximation of high-dimensional option pricing problems, *SIAM Journal on Scientific Computing* 29 (1) (2007) 440–458.
- [25] K. J. in 't Hout, S. Foulon, ADI finite difference schemes for option pricing in the Heston model with correlation, *International Journal of Numerical Analysis and Modeling* 7 (2) (2010) 303–320.
- [26] I. V. Oseledets, Tensor-train decomposition, *SIAM Journal on Scientific Computing* 33 (5) (2011) 2295–2317.
- [27] S. Mishra, R. Molinaro, Estimates on the generalization error of physics-informed neural networks for approximating PDEs, *IMA Journal of Numerical Analysis* 43 (1) (2023) 1–43.
- [28] T. De Ryck, S. Mishra, Error analysis for physics-informed neural networks (PINNs) approximating Kolmogorov PDEs, *Advances in Computational Mathematics* 50 (4) (2024) 83.
- [29] T. Takaishi, Rough volatility of Bitcoin, *Finance Research Letters* 32 (2020) 101379.
- [30] S. Liu, A. Borovykh, L. A. Grzelak, C. W. Oosterlee, A neural network-based framework for financial model calibration, *Journal of Mathematics in Industry* 9 (1) (2019) 9.